send a deactivation message that deactivates the wallet for several seconds.

With this deactivation, a user of the wallet doesn't have to remember to send an activation message to reactivate the wallet. When deactivated, the wallet sends an activation message to itself after a few seconds. How long, exactly, is specified in the previous deactivation message, which now contains the deactivation duration. Bear in mind that there is no waiting on the actor's side—the actor keeps processing messages as usual. This scheduling doesn't include anything like blocking.

Here's how this is represented in the protocol:

```
object WalletTimer {

  sealed trait Command
  final case class Increase(amount: Int) extends Command
  final case class Deactivate(seconds: Int) extends Command
  private final case object Activate extends Command

  ...
}
```

Apart from the deactivation, which now has the duration as a parameter, `Activate` is marked as `private`, so it is impossible to activate the wallet from outside the actor. The wallet can only be reactivated by the actor. This new variant is called `WalletTimer`.

NOTE Making the protocol private or public is a common pattern used in Akka. This allows you to provide both a public API, which is a set of messages that other actors can use, and a private API, which deals with the actor's internal functionality.

With this new wallet, you can create the application as follows: